

Shoogle

鞋類評價搜尋系統

專案 [Github](#) 連結

組別：週四 234 第九組

組員：113306025 陳心祈 113306033 黃子維 113306065 楊詠軒
113306085 何杰陞

1. Introduction — 主題與研究動機

1.1. 專案背景與核心問題

在當今的數位時代，消費者在做出購買決策前，習慣於透過網路搜尋引擎尋找相關評價。然而，對於鞋類這樣一個高度競爭的市場，資訊環境卻日益複雜。使用者面臨的核心困境包括：

- **資訊過載與商業噪音：** 搜尋結果往往被大量的電子商務網站、比價平台與廣告內容所淹沒。系統設計中明確定義的 `SHOPPING_DOMAINS`（如 `shopee.tw`, `momo.com.tw`）與 `SHOPPING_KEYWORDS`（如「購物」、「售價」、「下單」）等規則，正是為了識別並過濾這些佔據搜尋版面的商業性內容。
- **真實評價難以辨識：** 高品質、具深度的使用者原創內容（UGC）往往散落在部落格、論壇深處，難以與商業行銷文案區分。
- **搜尋效率低下：** 使用者需要手動瀏覽多個頁面，在大量無關資訊中費力地篩選出有價值的真實體驗分享，耗費了大量的時間與精力。

為了解決上述痛點，我們的 **Shoogle** 專案應運而生。其核心研究動機在於打造一個能自動化、智慧化地過濾商業噪音，並從廣大的網路中精準提取高品質、真實用戶評價的專門化搜尋引擎，藉此徹底優化使用者的消費決策流程。

1.2. 核心目標與價值主張

為實現上述願景，**Shoogle** 系統的設計圍繞以下三個核心目標：

- **高精準度評價識別：** 系統的首要任務是開發一套有效的演算法，能夠從海量網頁中，準確地將包含真實用戶評測的頁面與純商業銷售、廣告頁面區分開來。

- **搜尋結果關聯性：**系統不僅要找到評價內容，更要確保這些內容與用戶的原始搜尋意圖（如特定鞋款、品牌或風格）高度相關，提供真正有參考價值的資訊。
- **優化的使用者體驗：**在系統架構層面，我們採用非同步處理（`asynchronous processing`）模式。即使後端正在執行耗時的網路爬取與資料分析任務，前端介面依然能保持流暢與響應性，避免因長時間等待而導致的體驗下降。

綜上所述，Shoogle 系統的價值主張是：為使用者提供一個純淨、高效、可信賴的鞋類評價搜尋入口。透過智慧化的資訊過濾與排序，我們旨在節省使用者在資訊篩選上的時間成本，並為其提供更具洞察力的消費決策依據。

2. 系統架構、搜尋方法與評分公式

2.1. 系統架構與執行流程

本章節將全面剖析 Shoogle 系統的核心——「Rank Tricks」，即其獨特的搜尋與排序機制。這套機制是系統能否成功地從網路噪音中淘選出高價值資訊的關鍵所在。從使用者輸入查詢到最終結果呈現，數據在系統內部經歷了一系列精密的處理流程。

根據 `ApiMain` 的程式碼邏輯，完整的數據流可以概述如下：

1. **前端請求發起：**使用者在前端介面輸入關鍵字，觸發一個對後端 `GET /api/search-tree` 端點的請求。
2. **非同步任務啟動：**後端伺服器接收請求後，立即回應 `202 Accepted` 狀態碼，表示請求已被接受，並在背景線程池（`ExecutorService`）中啟動一個非同步的爬取與分析任務。

3. **智慧化查詢處理：**系統首先利用大型語言模型（LLM）對用戶輸入的關鍵字進行校正（`LlamaService.correctKeyword`）。
4. **並行化資料獲取：**系統透過 `Google Custom Search API`（`GoogleSearchApi`），並行地為不同的搜尋向量獲取一批高品質的初始網頁。`CompletableFuture` 機制確保了通用搜尋（`keyword + “鞋” + “評價”`）與論壇搜尋（`keyword + “鞋” + “ site:ptt.cc OR site:dcard.tw”`）可以同時進行。
5. **深度內容爬取與過濾：**針對獲取的種子 URL，系統使用 `Playwright` 函式庫（`PageScraper`）進行高擬真度的網頁內容爬取，並通過一套兩階段過濾策略，在爬取前後剔除大量無用連結。
6. **LLM 語意增強與核心評分：**系統再次調用 LLM 對核心關鍵字進行語意擴展（`LlamaService.expandKeywords`），生成一個相關詞彙集合。隨後，`ScoreCalculator` 模組基於這套擴展詞彙與一套精密的評分規則，為每個爬取到的頁面計算一個綜合分數。
7. **結果排序與快取：**所有頁面根據其計算出的分數進行降序排序，並將最終結果存儲在伺服器快取（`CACHED_RESULTS`，同時也會顯示在搜尋系統前端介面上的推薦搜尋關鍵詞）中。
8. **前端輪詢獲取：**前端介面透過定期輪詢 `GET /api/results` 端點，來檢查任務狀態並在任務完成後獲取最終排序好的結果列表。

2.2. 第一階段：智慧化查詢處理

遵循「Garbage In, Garbage Out」的原則，系統在正式執行搜尋前，必須對用戶的原始查詢進行一系列預處理。這一步是確保後續所有處理都建立在一個準確、高效基礎上的關鍵。

2.2.1. 非同步任務架構與 API 設計

為了提供流暢的使用者體驗，系統的后端 API 採用了非同步設計模式。長時間執行的爬取任務不會阻塞主 HTTP 請求線程。

- **GET /api/search-tree:** 此端點用於啟動一個新的搜尋任務。它接收使用者查詢後，立即將任務提交給 `ExecutorService` 執行，並返回 `202 Accepted`，告知前端任務已在背景處理。值得注意的是，採用 `GET` 方法來觸發一個狀態改變的背景進程。
- **GET /api/results:** 前端透過此端點輪詢任務狀態。回應中包含處理狀態（`processing` 或 `completed`）以及完成後的數據。
- **POST /api/stop:** 此端點允許使用者主動中斷正在執行的搜尋任務（例如使用者刷新網頁）。

系統透過 `IS_PROCESSING`, `SHOULD_STOP`, `currentTaskFuture` 等一系列 `volatile` 狀態變數，精準地管理著背景任務的生命週期。其並行模型被設計為單一任務處理器：在 `ApiMain` 中，若在舊任務執行期間收到新請求，系統將會強制中斷舊任務（`SHOULD_STOP = true; currentTaskFuture.cancel(true);`），然後再啟動新任務。此設計確保了系統資源的集中，但也意味著同一時間只會有一個搜尋請求被處理。

2.2.2. LLM 驅動的關鍵字校正

在搜尋流程的最前端，系統引入了大型語言模型（LLM）來校正用戶可能存在的輸入錯誤。`LlamaService.correctKeyword` 方法會將用戶的原始查詢發送給 `llama-3.3-70b-versatile` 模型（利用 `Groq API`）。將一個 LLM 調用置於流程的最開端，是一個深思熟慮的架構決策。它體現了「品質優先」的原則，即犧牲微小的啟動延遲，以換取後續所有步驟都基於一個最高品質的查詢，從而最大化地遵循「`Garbage In, Garbage Out`」原則。

其 `Prompt` 設計極為精簡，旨在引導模型執行單一、明確的任務：

你是一個搜尋引擎關鍵字修正助手。請檢查使用者的搜尋關鍵字「%s」是否有錯別字或語意不清。

規則：

- 1. 如果關鍵字有明顯錯別字（例如 'iphoe' -> 'iphone', 'addidas' -> 'adidas'），請回傳修正後的正確關鍵字。
- 2. 如果關鍵字看起來正確，請直接回傳原本的關鍵字。
- 3. 只回傳修正後的關鍵字，不要有任何解釋、標點符號或額外文字。

此策略的價值在於，它能在源頭上修正潛在的錯誤（如 'addidas' -> 'adidas'），確保後續的搜尋、爬取與評分都基於一個正確的目標，從而大幅提升了整個流程的有效性。

2.2.3. 多重搜尋向量策略

基於校正後的關鍵字，系統在 `ApiMain.runCrawlingTask` 中採用了雙軌並行的搜尋策略，以同時兼顧搜尋結果的廣度與深度。

搜尋種類	查詢字串範例	策略目標
通用評價搜尋	<code>keyword + "鞋" + "評價"</code>	廣泛抓取一般網站的評測文章。
論壇內容搜尋	<code>keyword + "鞋" + " site:ptt.cc OR site:dcard.tw"</code>	鎖定台灣兩大主流論壇 PTT 與 Dcard，專門獲取高價值的真實使用者生成內容（UGC）。

這種多重搜尋策略的優越性顯而易見：它將廣泛的網路內容掃描與對高價值社群的精準挖掘相結合，確保了最終搜尋結果既有多樣性，又包含來自真實使用者的可信聲音。

2.3. 第二階段：廣域資料獲取與過濾

在確定了精準的搜尋向量後，系統面臨的挑戰是如何高效地從網路上抓取原始資料，並在第一時間進行多層次的噪音過濾，以節省後續處理的計算資源。此階段的架構雖然支持遞歸式的深度爬取（`ContentExtractor.crawlRecursive`），但當前的營運配置（`maxDepthNormal = 0, maxDepthForum = 0`）將其限制為僅處理從 Google

獲取的種子 URL，這是一種專注於初始結果品質的策略，因子網頁常會抓取與關鍵字毫無關係的內容，同時也為未來的功能擴展保留了可能性。

2.3.1. 兩階段過濾管線：從源頭到爬取前的雙重防線

為了最大化資源效率，系統採用了一套兩階段的無用連結過濾管線，在不同處理階段剔除噪音。

1. **第一道防線（獲取時過濾）：**在 `GoogleSearchApi.java` 中，`_isExcluded` 方法扮演了第一道輕量級過濾器的角色。它在解析 Google API 回應時，會立即檢查 URL 是否包含 `facebook.com`, `youtube.com` 等明確的社交媒體域名或 `/login` 等功能性路徑。這一步確保了大量明顯無關的連結甚至不會被納入初始的種子 URL 列表，從源頭上減少了後續處理的負擔。
2. **第二道防線（爬取前過濾）：**`ContentExtractor.isJunkLink` 方法構成了更為全面和嚴格的第二道防線。在系統準備對一個 URL 發起昂貴的 Playwright 爬取操作之前，此方法會進行一次深度檢查。其過濾規則更為廣泛，涵蓋了更多類別：
 - **社交媒體平台：**如 `youtube.com`, `facebook.com`, `instagram.com`, `tiktok.com` 等。
 - **功能性頁面：**如 `/login`, `/cart`, `/checkout` 等與用戶操作相關的頁面。
 - **資訊性頁面：**如 `about`, `privacy`, `contact`, `sitemap` 等網站通用資訊頁面。

這套兩階段過濾策略展現了良好的資源管理哲學：先用低成本的方式快速排除最明顯的噪音，再用更全面的規則來把關計算密集型操作的入口。

2.3.2. 高擬真度網頁爬取

為了應對現代網站普遍採用的反爬蟲技術與動態網頁，甚至是懶加載（例如：Dcard 的留言，常會有使用者對鞋款的經驗感想），系統在 PageScraper.java 中利用了 Playwright 函式庫。選擇 Playwright 這樣一個完整的瀏覽器自動化函式庫，而非更簡單的 HTTP 客戶端（如 Jsoup），是一項關鍵的架構決策。它承認了現代網頁的現實——大量核心內容是通過客戶端 JavaScript 渲染的，因此，只有一個無頭瀏覽器才能確保捕獲到完整的 DOM 結構。其核心策略包括：

- **偽裝瀏覽器環境：** 透過設定標準的 User-Agent (Mozilla/5.0...)，並執行初始化腳本 `Object.defineProperty(navigator, 'webdriver', {get: () => undefined});`，系統能夠有效地隱藏其作為自動化工具的特徵，降低被網站偵測的風險。
- **資源請求攔截：** `context.route` 機制被用來攔截對圖片、字體、CSS 等非核心內容資源的網路請求。直接終止這些請求極大地提升了頁面載入速度，並顯著降低了爬取過程中的頻寬與記憶體消耗。
- **智慧等待機制：** 系統針對不同類型的網站採用了定製化的等待策略。對於 Dcard 這類大量使用前端動態渲染技術的網站，`handleDcardSmartWait` 會模擬使用者向下滾動頁面，以觸發更多內容的非同步載入。而對於常規網站，`handleGeneralSmartWait` 則會等待網路請求達到空閒狀態，確保頁面元素已完全載入。

2.3.3. 內容萃取與純淨化

在成功獲取頁面 HTML 後，`PageScraper.parseHtml` 方法利用 Jsoup 函式庫進行高效的內容解析。此階段的關鍵任務是「內容純淨化」。透過執行以下程式碼：`body.select("script, style, noscript, iframe, svg, button, form, .ad, header, footer, nav").remove();` 系統能夠精準地移除所有與核心正文無關的元素，如腳本、樣式表、導航列、頁首、頁尾以及廣告區塊。這一步驟確保了後續的評分演算法只處理頁面的核心文本內容，排除了大量無關噪音的干擾。

2.3.4. Playwright 的缺陷

雖然使用 `Playwright` 更容易去爬取完整的網頁內容，但弊端也顯而易見，對一個網頁來說，從 `PageScraper.parseHtml` 方法到利用 `Jsoup` 函式庫解析完整內容大約需要 8 到 9 秒，整個搜尋流程（共 12 個網頁，以對半分，非同步執行方式運作）大約需要近一分鐘的搜尋時間，對於整個系統、使用者體驗來說必是一大負擔。

2.4. 第三階段：語意增強與評分計算

此階段堪稱 `Shoogle` 系統的「大腦」。在獲取了純淨的網頁文本後，系統將利用 LLM 的語意理解能力和一套精密的評分演算法，對資訊進行深度理解與價值判斷，最終實現搜尋結果的精準排序。

2.4.1. LLM 驅動的語意擴展

為了讓評分系統能夠理解超越字面匹配的相關概念，

`LlamaService.expandKeywords` 方法被用來進行語意擴展。它將用戶的單一關鍵字（如 "Puma Speedcat"）提交給 LLM，並透過以下精心設計的 Prompt，引導模型扮演「專業的球鞋分析師」：

你是一個專業的球鞋分析師。請列出 5 個與鞋類關鍵字「%s」高度相關的同義詞、型號暱稱、相關名人或風格描述。

規則：

1. 只回傳關鍵詞，不要有任何解釋或開場白。

範例輸入：Puma Speedcat

範例輸出：賽車鞋,薄底鞋,Rosé 同款,復古,Blackpink

此步驟的關鍵影響在於，它將一個關鍵字擴展為一個語意集合（如 賽車鞋, 薄底鞋, Rosé 同款, Blackpink）。這使得評分模型能夠識別和獎勵那些雖然沒有直接提及 "Puma Speedcat"，但討論了與其高度相關概念的內容，從而大幅提升了搜尋的召回率與準確性。此外，`KEYWORD_CACHE` 的實作不僅是性能優化，更是一項關

鍵的成本控制措施，它避免了對昂貴的 llama-3.3-70b-versatile 模型進行重複 API 調用，確保了系統的經濟可行性。

2.4.2. 核心評分演算法：ScoreCalculator

ScoreCalculator.calculate 方法是整個排序機制的核心，其邏輯可以分為「排除性負向評分」和「累積性正向評分」兩大部分。

2.4.2.1. 排除性負向評分：商業內容懲罰

為了徹底過濾商業噪音，系統建立了一道雙層防火牆：

懲罰機制	觸發條件	分數影響	程式碼依據
硬性排除	URL 的域名或路徑匹配預定義的購物網站列表。	-1000.0	isShoppingUrl(), SHOPPING_DOMAINS, SHOPPING_URL_PATHS
軟性排除	採用雙軌制檢查：(1) 頁面標題包含任何一個購物關鍵字；或 (2) 頁面內容中，與購物相關的詞彙加權計數達到 5 分（例如，「加入購物車」計 5 分，「庫存」計 2 分）。	-500.0	isShoppingPattern(), SHOPPING_KEYWORDS

這套雙層過濾系統設計精巧：硬性排除能夠快速、低成本地剔除所有已知的電商平台；而軟性排除則通過對標題的快速掃描和對內容的加權分析，進一步捕捉到那些可能偽裝成內容網站，但實質為導購或銷售的頁面。兩者結合，構成了強大的商業內容過濾屏障。

2.4.2.2. 累積性正向評分：關聯性與評價指標

對於通過了商業內容過濾的頁面，系統會根據以下三個加權因子計算其正向分數：

1. **基礎關鍵字關聯性分數：** 原始關鍵字以及所有由 LLM 擴展出的關鍵字，在頁面內容中每出現一次，分數增加 1.0。這構成了頁面內容與用戶查詢相關性的基礎分。
2. **標題加權分數：** 如果原始關鍵字出現在頁面的 <title> 標籤中，將額外獲得 5.0 的加分。此權重基於一個普遍假設：標題內容更能精準地反映文章的核心主題。
3. **核心評價詞彙加權分數：** 此為整個評分系統的精髓所在。系統內建了一個 EVALUATION_KEYWORDS 列表，包含了大量描述穿著體驗的正面與負面詞彙，如「好穿」、「舒適」、「踩屎感」、「磨腳」、「太硬」等。這些詞彙在內容中每出現一次，即可獲得高達 10.0 的權重。此設計使得系統能夠量化一篇內容的「評價濃度」，那些真正深入討論使用者穿著感受、提供具體體驗細節的文章，將因此獲得極高的分數，從而在排序中脫穎而出。

2.4.3. 最終排序與結果呈現

在所有頁面的分數計算完成後，ApiMain 中的 `allTrees.sort(Comparator.comparingDouble(WebTree::getScore).reversed());` 程式碼會對所有 WebTree 物件，根據其 `score` 屬性進行降序排序。

這個經過智慧排序的最終結果列表，被存儲在快取中，並透過 `GET /api/results` 端點提供給前端。至此，從使用者的一個簡單查詢開始，到呈現一份經過深度過濾、語意理解與價值排序的精選評價報告，整個搜尋閉環宣告完成。Shoogle 系統透過這套多階段、多層次的智慧化處理流程，成功地將資訊檢索轉化為知識發現。

3. Schedule — 專案進度規劃

3.1 緒論：從藍圖到實現的開發路徑

3.2 邏輯開發階段

第一階段：基礎架構與資料模型定義

- **主要目標：**搭建專案的技術骨架，定義核心的資料結構，為後續所有功能模組的開發奠定穩固的基礎。
- **交付成果：**
 - 建立 **ApiMain** 類別，並使用 **Spark Java** 成功啟動一個能響應基本 HTTP 請求的 Web 伺服器。
 - 定義 **WebTree** 類別，確立其作為系統中資訊流動的標準化載體，包含 URL、標題等基礎屬性。
- **階段性意義：**此階段確立了專案的技術棧（Java, Spark Java）與最根本的資料模型。它好比建築的地基，雖然看不見華麗的功能，卻是支撐整個系統穩定運作的關鍵。

第二階段：外部資料源整合

- **主要目標：**打通系統與外部世界的聯繫，使其具備從網際網路獲取初步資料（種子 URL）的能力。
- **交付成果：**
 - 開發 **GoogleSearchApi** 模組，成功封裝並串接 Google Custom Search API。
 - **ApiMain** 能夠根據使用者輸入的查詢詞，調用 **GoogleSearchApi** 並獲取一組相關的 URL 列表。
- **階段性意義：**此階段標誌著系統從一個封閉的內部應用，轉變為一個能夠與外部數據源互動的系統。它驗證了資料擷取管道第一環節的可行性。

第三階段：高保真網頁內容提取

- **主要目標：**攻克現代網頁普遍存在的 JavaScript 動態渲染問題，確保能夠獲取到真實、完整的網頁全文內容，而非僅僅是靜態 HTML 原始碼。
- **交付成果：**
 - 開發 PageScraper 模組，成功整合 Playwright 函式庫來驅動無頭瀏覽器。
 - 實現了包括偽裝使用者代理（User-Agent）、攔截非必要資源請求在內的反爬蟲與效能優化策略。
- **階段性意義：**這是專案在技術層面最重大的突破之一。成功實現高保真內容提取，確保了後續所有內容分析與評分演算法所依賴的資料是高品質且完整的。

第四階段：爬蟲流程協調與非同步化

- **主要目標：**將前序階段開發的獨立工具（URL 獲取、內容提取）整合為一個自動化的工作流程，並將其改造為非同步模式以應對長時間運行的任務。
- **交付成果：**
 - 開發 ContentExtractor 類別，用於協調 GoogleSearchApi 和 PageScraper，並實現了關鍵的 isJunkLink 預過濾邏輯。
 - 在 ApiMain 中引入 ExecutorService，將整個爬取流程（runCrawlingTask）置於背景執行緒中處理。
- **階段性意義：**系統至此從一系列獨立的功能模組，演變為一個可實際運作、自動化的資料處理管道。非同步化的實現，使其具備了成為一個真正可用的網路服務的基礎。

第五階段：核心排名演算法的實現

- **主要目標：**為系統注入靈魂——賦予其分辨「優質評價」與「商業推銷」資訊的核心商業邏輯。

- 交付成果：
 - 開發 ScoreCalculator 模組。
 - 定義並實現了一套基於多維度規則的複雜計分演算法，包括對購物網站的強力懲罰機制和對評價性詞彙的高權重獎勵。
- 階段性意義：這是 Shoogle 系統核心價值的直接體現。從此，它不再是一個簡單的網頁爬蟲，而是一個具備特定領域判斷能力的專業評價篩選工具。

第六階段：AI 智慧層的注入

- 主要目標：引入大型語言模型（LLM）的語意理解能力，以克服傳統基於關鍵字匹配的局限性，讓系統更「聰明」。
- 交付成果：
 - 開發 LlamaService 模組，成功串接 Groq API。
 - 將關鍵字修正（correctKeyword）與關鍵字擴展（expandKeywords）功能無縫整合至 runCrawlingTask 的主流程中。
- 階段性意義：AI 的引入為系統增加了智慧化的維度。它使得系統更能理解使用者的真實意圖（修正錯別字），並能發現更多隱蔽但高度相關的內容（擴展同義詞），極大地提升了搜尋的精準度與召回率。

第七階段：系統健壯性與最終整合

- 主要目標：完善系統的穩定性與使用者體驗，處理各種邊界情況，並對所有模組進行端對端的整合測試。
- 交付成果：
 - 在 ApiMain 中實現了完整的任務停止機制（/api/stop 端點與 SHOULD_STOP 旗標），允許使用者中斷長時間運行的任務。
 - 建立了 CACHED_RESULTS 結果快取機制，並完善了 /api/results 的輪詢邏輯。

- 完成了所有模組的最終整合與測試，確保資料流在各個環節都能順暢流動。
- **階段性意義：**標誌著專案核心開發階段的完成。系統在此階段已達到功能完備、流程穩定、可供部署與交付的成熟狀態。

4. 系統成效分析（配合下面附錄圖片）

本章節將針對 **Shoogle** 系統的實際執行表現進行深度剖析。我們將從搜尋品質、資訊純淨度以及技術成本三個維度，將本系統與傳統搜尋引擎（如 **Google**）及生成式人工智慧（**Gemini**）進行比較，並坦誠面對當前系統在效能上的瓶頸。

4.1 與 **Google** 搜尋引擎之優劣對比

Google 作為全球最大的搜尋引擎，其核心優勢在於極致的檢索速度與海量的索引資料。然而，對於「鞋類評價」這一垂直領域，**Google** 的搜尋結果往往面臨以下問題：

- **電商與廣告過載：**當使用者在 **Google** 搜尋「Puma Speedcat 評價」時，前幾個頁面往往充斥著 **Shopee**、**Momo**、**PChome** 等購物平台。這些平台雖有評價機制，但多為「出貨快速」等物流評價，而非針對鞋款性能的深度分析。
- **SEO 過度優化：**許多營利性網站透過 **SEO** 技術佔據高位，內容卻是重複性高的行銷辭令。

Shoogle 的核心優勢：

- **專業的情境過濾：**系統內建了 **SHOPPING_DOMAINS** 黑名單，能自動屏蔽包括 **shopee.tw**、**amazon.com** 等 19 個主流電商平台。

- **深度的內容計分：**與 Google 僅依靠 PageRank 不同，我們的 ScoreCalculator 會針對「踩屎感」、「磨腳」、「版型偏大」等 26 個鞋類專屬評價關鍵字進行加權計算。
- **精準的關鍵字修正：**透過 LlamaService.correctKeyword，我們能修正使用者的錯別字（如將 "addidas" 修正為 "adidas"），這比 Google 的「您是不是要查」更具強制性與精準度。

4.2 與大型語言模型 (LLM) 之優劣對比

直接使用 ChatGPT 或 Llama 進行鞋類諮詢是目前許多使用者的選擇，但其缺點在於：

- **幻覺問題 (Hallucination)：**LLM 可能會憑空捏造某雙鞋的腳感，或混淆不同型號的規格。
- **資料時效性：**LLM 的訓練資料有截止日，對於最新發售的聯名鞋款，LLM 無法提供即時的論壇討論。

Shoogle 的混合優勢：

- **即時性與真實性：**Shoogle 結合了 LLM 的理解能力與實時爬蟲技術。我們利用 LLM 進行「關鍵字擴展」，但最終呈現給使用者的是真實存在的網頁連結與即時內容。
- **證據導向：**每一個評分都有來源網頁支持，使用者可以點擊連結確認內容，而非盲目相信 AI 生成的總結。

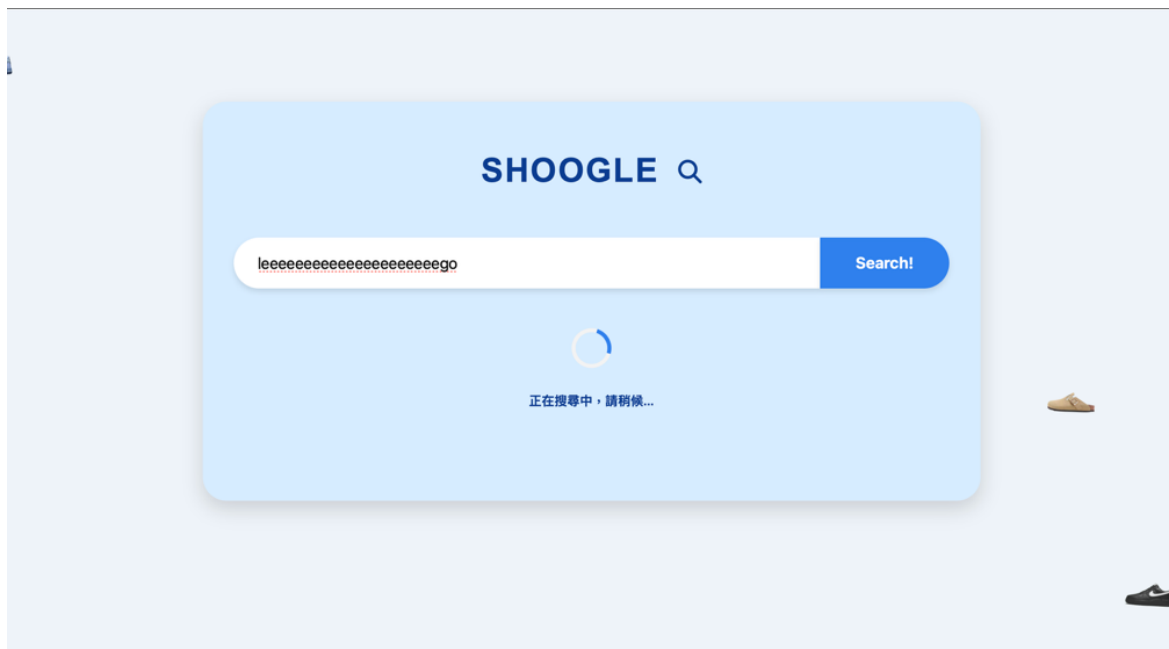
4.3 效能挑戰：Playwright 的「雙面刃」

目前系統完成一次深度搜尋大約需要 **60 秒**。這主要是由系統架構中選擇使用 Playwright 驅動瀏覽器所導致的技術權衡：

- **動態渲染成本：**現代論壇（如 Dcard）採用 SPA 技術。為獲取內容，系統必須啟動 Chromium 實例並模擬滾動行為以觸發懶加載。
- **反爬蟲對抗：**PageScraper 必須偽造 User-Agent 並隱藏 navigator.webdriver 屬性，這些操作保證了成功率，卻增加了初始化時間。
- **串行與併行限制：**受限於記憶體與執行緒池 (FixedThreadPool(4))，無法同時開啟過多瀏覽器視窗。
- **智慧等待時間：**handleDcardSmartWait 中的強制等待與滾動機制是不可省略的必要開銷。

結論：這是一次「以時間換取深度」的技術決策。雖然 Google 只要 0.5 秒，但使用者可能需花 10 分鐘過濾垃圾；Shoogle 雖需等待 60 秒，但呈現的是高度過濾後的純淨資訊。

附錄 1：Shoogle 搜尋“leeeeeeeeeeeeeeeeeeeeeego”，嘗試拼字錯誤尋找 lego 鞋款評價



SHOOGLE 🔍

leeeeeeeeeeeeeeeeeeeego

Search!

已自動修正關鍵字為: lego

樂高 積木鞋 兒童鞋 玩具風 彩色鞋款 lego

[出售] 全國Adidas樂高鞋、ralph lauren褲子- 看板BabyProducts - 批 ...
www.ptt.cc

批踢踢實業坊，看板 BabyProducts 關於我們 聯絡資訊 返回看板 作者lipano (Melody bunny) 看板 BabyProducts 標題[出售] 全國 Adidas樂高鞋、ralph lauren褲子 時間Sat Dec 28 14:43:55 2024 1. 商品名稱：A...

32.0分



新聞分享/ LEGO x Nike 首款實鞋終於來了！鞋盒設計還另有用途 ...
kenlu.net

新聞分享 / LEGO x Nike 首款實鞋終於來了！鞋盒設計還另有用途 醜態超級久 By Kai On May 27, 2025 4,556 2024 年 8 月，Nike 官方正式宣佈與 LEGO 合作，並預告會有許多聯名作品出現，而在快一年的等待後，第一款 LEGO x Nike 實鞋終於囉...

22.0分



www.dcard.tw

註冊 / 登入 襯衫變男 個人牆 推薦 可愛又實用！【樂高LEGO】正版授權積木造型兒童背包，可當多層收納電腦包 襯衫變男 23/10/5 11:58 之前在好市多Costco看到這個背包，超可愛的啊！這個包的做工很不錯，而且滿有質感 後來研究，原來之前7-11也有出過(集點加購的那種)【樂高...

21.0分



[Lego] ADIDAS X LEGO 分享- 看板Toy - 批踢踢實業坊
www.ptt.cc

批踢踢實業坊，看板 Toy 關於我們 聯絡資訊 返回看板 作者cookie0215 () 看板Toy 標題[Lego] ADIDAS X LEGO 分享 時間Tue Sep 8 10:56:14 2020 https://youtu.be/j9A7lbSd518 不知道LEGO的鞋盒會不會是公開...

11.0分



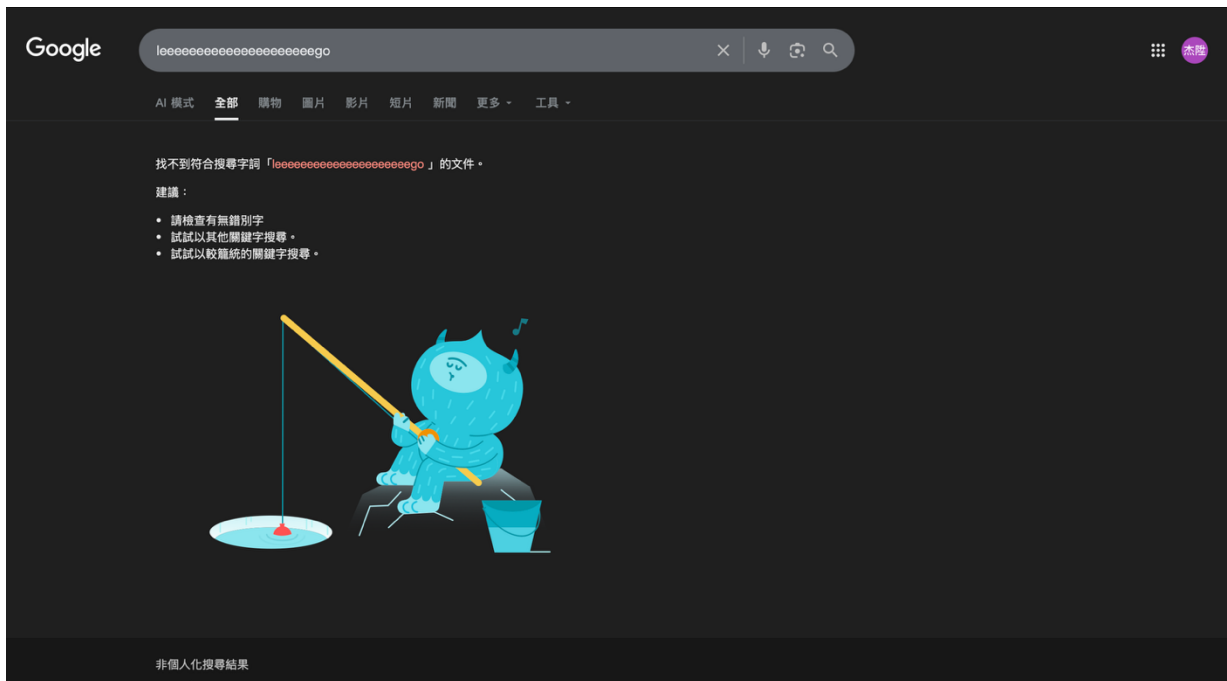
D+AF女鞋 | 好穿一直穿，柔軟舒適不將就—人氣女鞋第一品牌
www.daf-shoes.com

使用APP，購物更流暢！滿2500現折\$250 >> 全部商品 新品TOP. 50女鞋靴子門市資訊最新消息流行話題 0 常見問題Q&A login 立刻結帳 新品上市 本週新上架 人氣TOP.50 2025美靴新品 現貨不用等 特價活動 聖誕治鞋款999up 3D零感雲纖鞋9折 MIT國產鞋9折...

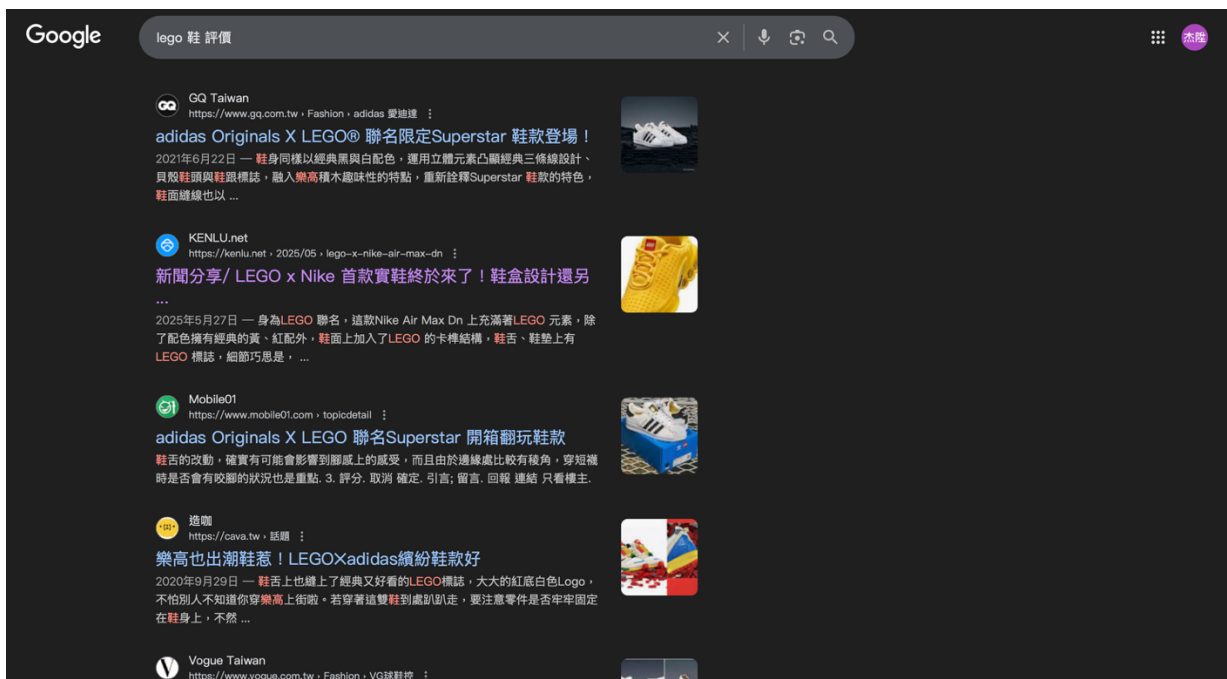
10.0分



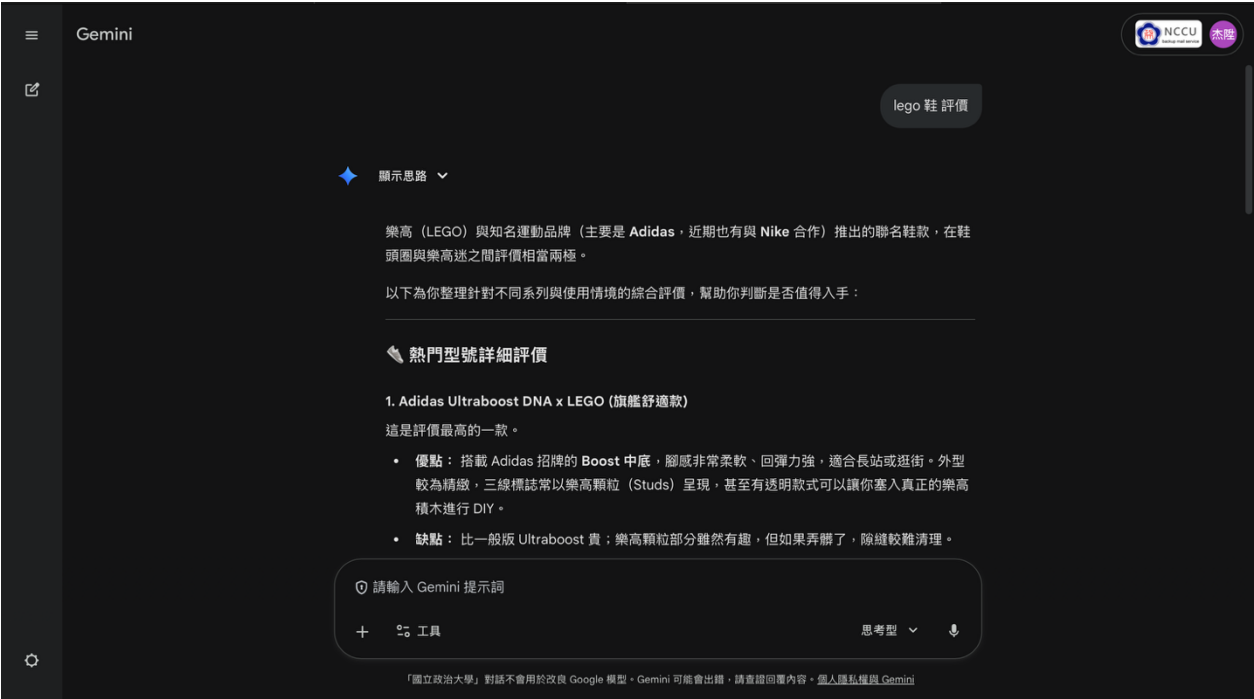
附錄 2：Google 搜尋”leeeeeeeeeeeeeeeeeeeeeego”，嘗試拼字錯誤尋找 lego 鞋款評價



附錄 3：Google 搜尋”lego 鞋 評價”，嘗試尋找 lego 鞋款評價



附錄 4：Gemini 詢問”lego 鞋 評價”，嘗試尋找 lego 鞋款評價



5. 未來展望與結語

Shoogle 目前已成功建立了一個從關鍵字修正、多源搜尋、內容爬取到數據結構化計分的完整鏈條。

5.1 短期改進：效能優化 (Efficiency)

針對延遲問題，我們計劃採取以下技術手段：

- **分散式爬蟲集群**：將 Playwright 實例部署在多台伺服器上，實現真正的分佈式抓取。
- **Headless 預熱機制**：在背景維持一定數量的瀏覽器 Context，減少啟動時間損耗。
- **混合爬取策略**：針對靜態網站切換為極速的 Jsoup 抓取，僅針對動態網站保留 Playwright 模式。

5.2 中期發展：數據結構的深度利用 (Data Structures)

- **廣度優先遍歷 (BFS)**：未來將開啟 crawlRecursive 功能，從優質評價網頁延伸抓取其推薦連結，建立「鞋類知識圖譜」。
- **持久化緩存**：引入 Redis 或向量資料庫，儲存已爬取的文章與關鍵字向量，實現「秒級」的二次查詢。

5.3 長期願景：多模態與情感分析 (AI Integration)

- **視覺評價分析**：結合視覺模型分析網頁圖片，自動辨識鞋款的磨損程度或實穿外型。
- **情緒極性分析**：引入 NLP 模型判別評論是「真心推薦」還是「反諷抱怨」，提供正負評比例分布圖。

5.4 結語

Shoogle 的開發不僅是資料結構與演算法的實踐，更是對「資訊爆炸卻資訊貧瘠」現況的一種反思。透過結合 Google Search API 的廣度、Playwright 的深度以及 Llama 3.3 的理解力，我們證明了開發者有能力從混亂的網路環境中，利用自定義計分演算法提煉出真正有價值的決策依據。本專案作為技術驗證，成功實現了從搜尋到評分的自動化流程，為鞋類評價的垂直搜尋系統奠定了堅實的基礎。